

# Fase 1 — Webcam que segue o rosto · Plano de implementação

---

Protótipo 100% no PC (Python), dirigindo o ElectronBot via a DLL C-ABI do pacote AHK. Sem compilar C++, sem mexer em firmware.

## 1. Escopo e princípio

---

- **Eixo mecânico real:** só **horizontal** (cintura = junta **j6**,  $\pm 90^\circ$ ). A câmera está no tronco; girar a cintura gira a câmera.
- **Eixo vertical:** **não** é mecânico (nenhuma junta inclina a câmera) → tratado por **crop digital** na saída.
- **Saída:** feed re-centralizado exposto como **webcam virtual** (OBS Virtual Camera) para Zoom/Teams/OBS.
- **Display:** mostra olhos estáticos (cada `Sync` reenvia a mesma imagem — ver C1 do roadmap).

## 2. Dependências

---

```
pip install opencv-python numpy pyvirtualcam
```

- **OBS Studio** instalado (fornece o backend "OBS Virtual Camera" que o `pyvirtualcam` usa no Windows).
- **DLLs do robô** (as do pacote AHK, que exportam `AHK_*`): `3.Software/_Tools/AHK-ExpansionPack/ElectronBotSDK/` → `ElectronBotSDK-LowLevel.dll`, `USBInterface.dll`, `opencv_world455.dll`, `libusb0.dll`.
- **Pré-requisito (Fase 0):** driver via Zadig/BotDriver + servos calibrados (ServoToolKit).

⚠ As DLLs do robô são **x86 (32-bit)**? Verificar a arquitetura e usar um **Python da mesma bitness** para o `ctypes` casar. (Checar com `dumpbin /headers` ou tentativa de `LoadLibrary .`)

## 3. Estrutura de arquivos

---

```

webcam_follow/
├─ main.py           # loop principal
├─ robot.py          # binding ctypes -> AHK_* (camada de hardware)
├─ tracker.py        # captura + detecção/seguimento de rosto
├─ controller.py     # malha de controle (erro -> yaw j6)
├─ vcam.py           # saída p/ webcam virtual + crop vertical
├─ config.py         # parâmetros (ganhos, limites, paths, VID/PID)
├─ assets/eyes.jpg   # imagem 240x240 dos olhos p/ o display
└─ dll/              # cópia das DLLs do robô (mesma pasta)

```

## 4. robot.py — binding ctypes

```

import ctypes, os

class Robot:
    def __init__(self, dll_dir, dll_name="ElectronBotSDK-LowLevel.dll"):
        os.add_dll_directory(dll_dir)          # acha USBInterface/opencv/libusb
        self.lib = ctypes.CDLL(os.path.join(dll_dir, dll_name))
        # assinaturas
        self.lib.AHK_New.restype = ctypes.c_void_p
        self.lib.AHK_Connect.argtypes = [ctypes.c_void_p]; self.lib.AHK_Connect.restype = ctypes.c_bool
        self.lib.AHK_Disconnect.argtypes = [ctypes.c_void_p]
        self.lib.AHK_Sync.argtypes = [ctypes.c_void_p]; self.lib.AHK_Sync.restype = ctypes.c_bool
        self.lib.AHK_SetImageSrc_Path.argtypes = [ctypes.c_void_p, ctypes.c_char_p]
        self.lib.AHK_SetJointAngles.argtypes = [ctypes.c_void_p] + [ctypes.c_float]*6 + [ctypes.c_bool]
        self.lib.AHK_GetJointAngles.argtypes = [ctypes.c_void_p, ctypes.POINTER(ctypes.c_float)]
        self.p = self.lib.AHK_New()

    def connect(self):    return self.lib.AHK_Connect(self.p)
    def disconnect(self): self.lib.AHK_Disconnect(self.p)

    def set_eyes(self, path):          # imagem do display
        self.lib.AHK_SetImageSrc_Path(self.p, path.encode("ascii"))

    # ORDEM RAW: j1=cabeça, j2..j5=braços, j6=cintura(giro)
    def set_pose(self, head=0., body=0., enable=True):
        self.lib.AHK_SetJointAngles(self.p, head, 0., 0., 0., 0., body, enable)

    def sync(self):          return self.lib.AHK_Sync(self.p)    # envia frame + juntas

    def get_joints(self):          # precisa 2 leituras (C2)
        buf = (ctypes.c_float*6)()
        self.lib.AHK_GetJointAngles(self.p, buf)
        return list(buf)

```

## 5. tracker.py — captura + rosto

```

import cv2

class FaceTracker:
    def __init__(self, cam_index=0):
        self.cap = cv2.VideoCapture(cam_index, cv2.CAP_DSHOW)
        self.det = cv2.CascadeClassifier(
            cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
        # v2: trocar por detector DNN (res10 SSD) p/ robustez

    def read_upright(self):
        ok, frame = self.cap.read()
        if not ok: return None
        return cv2.rotate(frame, cv2.ROTATE_180) # desfaz montagem 180° (C4)

    def biggest_face(self, frame):
        g = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        faces = self.det.detectMultiScale(g, 1.2, 5, minSize=(60,60))
        if len(faces) == 0: return None
        x,y,w,h = max(faces, key=lambda f: f[2]*f[3]) # maior rosto
        return (x + w/2, y + h/2, w, h)

```

## 6. controller.py — malha de controle (erro → j6)

Controle de **velocidade** (incremental): a cintura gira a uma taxa proporcional ao erro horizontal até centralizar. Robusto sem precisar de mapeamento pixel→grau exato.

```

class YawController:
    def __init__(self, cfg):
        self.cfg = cfg
        self.yaw = 0.0 # comando atual (graus)
        self.err_ema = 0.0

    def update(self, face_cx, frame_w):
        if face_cx is None: # sem rosto -> volta devagar ao centro
            self.yaw += (0 - self.yaw) * 0.02
            return self._clamp(self.yaw)

        err = (face_cx - frame_w/2) / (frame_w/2) # [-1, 1]
        if abs(err) < self.cfg.deadzone: err = 0.0
        self.err_ema = (self.cfg.ema*err) + (1-self.cfg.ema)*self.err_ema

        step = self.cfg.kp * self.err_ema * self.cfg.sign # graus/tick
        step = max(-self.cfg.max_rate, min(self.cfg.max_rate, step)) # rate-limit
        self.yaw = self._clamp(self.yaw + step)
        return self.yaw

    def _clamp(self, v):
        lim = self.cfg.yaw_limit
        return max(-lim, min(lim, v))

```

config.py (valores iniciais p/ tuning):

```

class Cfg:
    kp = 6.0; ema = 0.4; deadzone = 0.06
    max_rate = 4.0          # graus por tick
    yaw_limit = 85.0        # margem dos ±90
    sign = +1              # determinar na 1ª execução (qual lado gira)
    eyes_path = "assets/eyes.jpg"
    dll_dir = r"dll"

```

**Calibração do `sign`** : na primeira execução, com o rosto à direita, ver se a cintura gira no sentido que **centraliza**. Se afastar, inverter `sign`.

---

## 7. `vcam.py` — saída + crop vertical

---

```

import pyvirtualcam, cv2

class VCam:
    def __init__(self, w, h, fps=30):
        self.cam = pyvirtualcam.Camera(width=w, height=h, fps=fps)
        self.w, self.h = w, h

    def push(self, frame, face_cy=None):
        if face_cy is not None:          # re-centra verticalmente (digital)
            shift = int(self.h/2 - face_cy)
            M = [[1,0,0],[0,1,shift]]
            frame = cv2.warpAffine(frame, __import__("numpy").float32(M),
                                   (self.w, self.h))
        self.cam.send(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
        self.cam.sleep_until_next_frame()

```

---

## 8. `main.py` — loop

---

```

from robot import Robot; from tracker import FaceTracker
from controller import YawController; from vcam import VCam; from config import Cfg

cfg = Cfg()
bot = Robot(cfg.dll_dir)
assert bot.connect(), "Robô não conectou (checar Zadig/driver)"
bot.set_eyes(cfg.eyes_path)

tr = FaceTracker(0)
ctl = YawController(cfg)
probe = tr.read_upright(); H, W = probe.shape[:2]
vc = VCam(W, H, 30)

try:
    while True:
        frame = tr.read_upright()
        if frame is None: continue
        face = tr.biggest_face(frame) # (cx,cy,w,h) ou None
        cx = face[0] if face else None
        yaw = ctl.update(cx, W)
        bot.set_pose(head=0.0, body=yaw, enable=True)
        bot.sync() # envia olhos + juntas
        vc.push(frame, face[1] if face else None)
finally:
    bot.set_pose(0,0, enable=True); bot.sync()
    bot.disconnect()

```

## 9. Comportamentos e bordas

| Situação               | Tratamento                                                                                              |
|------------------------|---------------------------------------------------------------------------------------------------------|
| Sem rosto              | Cintura volta devagar ao centro (não trava no canto)                                                    |
| Vários rostos          | Segue o <b>maior</b> ( $\approx$ mais próximo). v2: lock por reconhecimento                             |
| Oscilação              | Aumentar <code>deadzone</code> / <code>ema</code> , reduzir <code>kp</code> / <code>max_rate</code>     |
| Brownout (servo + USB) | Só mover <b>j6</b> nesta fase; evitar movimentos bruscos                                                |
| Privacidade            | Tecla de "parque" → <code>body=0</code> , olhos fechados, parar vcam                                    |
| Hot-plug               | Envolver <code>connect()</code> em <code>retry</code> ; detectar <code>sync()</code> falho → reconectar |

## 10. Critérios de aceite (DoD)

1. Mover o rosto na horizontal → a cintura acompanha e **centraliza** (latência percebida < ~300 ms).
2. Saída aparece como câmera "OBS Virtual Camera" e é selecionável no Zoom/Teams.
3. Re-centralização vertical digital mantém o rosto na faixa central da saída.
4. Sem rosto → robô estabiliza no centro, sem oscilar.
5. Encerrar limpo (volta ao centro, desconecta).

## 11. Riscos específicos da fase

- **Bitness DLL x Python** (provável x86) — usar Python da mesma arquitetura.
- **C1**: cada `sync()` manda frame inteiro; manter taxa ~20–30 Hz e olhos estáticos.
- **C4**: rotação 180° já tratada no `read_upright`; conferir também a saída do vcam.
- `sign` indefinido até a 1ª calibração.
- **OBS Virtual Camera** precisa estar instalada/registrada.

## 12. Evolução (v2+)

---

- Detector **DNN** (res10 SSD) ou MediaPipe p/ robustez a ângulo/iluminação.
- **Tracker** (KCF/CSRT) entre detecções p/ suavidade.
- Usar **head pitch (j1,  $\pm 15^\circ$ )** para microajuste de "atenção" (não move a câmera, mas dá vida).
- Mostrar no display uma reação (olhar na direção da pessoa) em vez de olhos estáticos.